

Time series clustering

Julien JACQUES

Introduction

Basics of Time Series Clustering

To go further

Features extraction from time series

Introduction

Clustering

The **goal of clustering** is to create homogeneous group of observations, s.t.:

- ▶ observations within a group are as similar as possible
- ▶ groups are as different as possible from each other

The groups are called **clusters**.

Use of clustering

- ▶ Clustering is an unsupervised technique.
- ▶ It aims to explore the data and to discover some typical pattern.
- ▶ It is often used as a preliminary step between supervised approach.

The data

Our goal is to cluster time series.

For instance, the number of new cases of Covid19 in the different countries of the world.

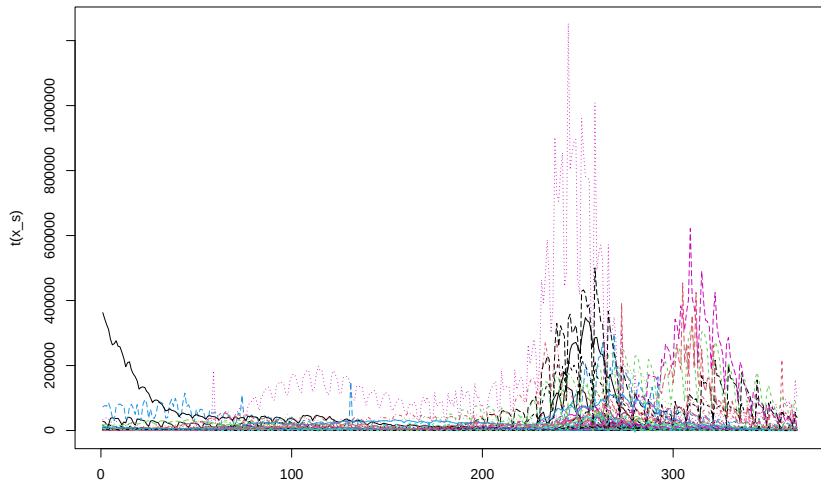
```
covid19 = read.csv("data/2022-05-13-WHO-COVID-19-global-dat
x=matrix(covid19$New_cases,ncol = 861,byrow = TRUE)
rownames(x)=unique(covid19$Country_code)
x_s=x[, (861-364):861]
x_s=x_s[rowMeans(x_s)>1000,]
```

For the example, a subset of countries having large number of cases are selected

The data

We have 80 time series, observed on 365 points

```
matplot(t(x_s),pch=20,type='l')
```



We want to cluster them into homogeneous group

Basics of Time Series Clustering

Time serie clustering

An easy way is to use usual algorithms:

- ▶ kmeans,
- ▶ hierarchical clustering,

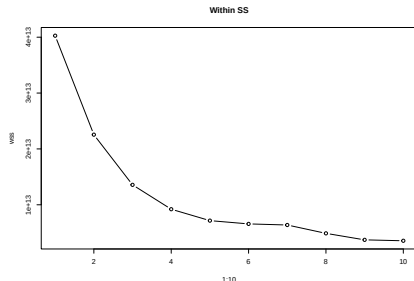
applied on a given distance for time series:

- ▶ either the usual Euclidean distance
- ▶ or specific distances as DTW

Kmeans with Euclidean distance

We can apply the usual kmeans algorithm using the Euclidean distance between time series:

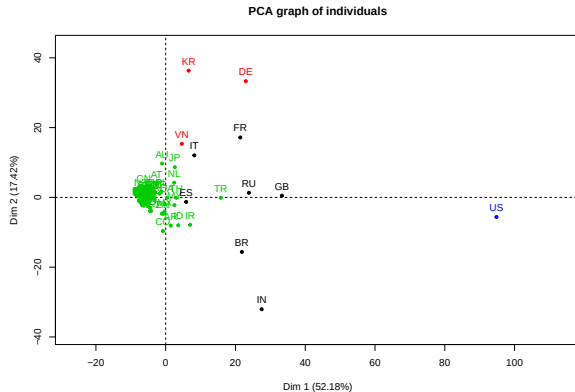
```
wss=NULL
for (k in 1:10){
  tmp=kmeans(x_s,centers=k)
  wss=c(wss,tmp$tot.withinss)
}
plot(1:10,wss,main="Within SS",type="b")
```



May be 4 clusters ?

Kmeans with Euclidean distance

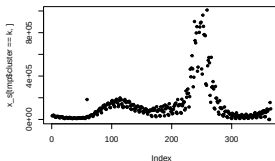
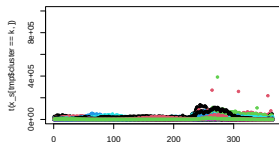
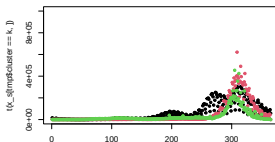
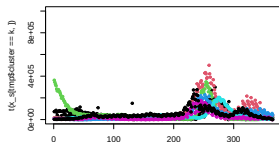
```
tmp=kmeans(x_s,centers=4)
library(FactoMineR)
pca<- PCA(x_s, graph = FALSE)
plot(pca,choix = "ind",col.ind = tmp$cluster,
      graph.type = "classic")
```



Kmeans with Euclidean distance

Representation of the curves per cluster

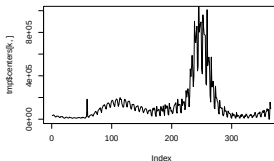
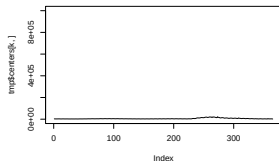
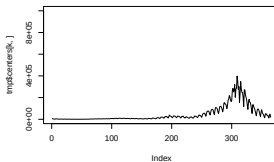
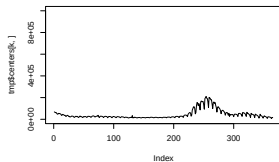
```
par(mfrow=c(2,2))
for (k in 1:4){
  if (tmp$size[k]>1){matplot(t(x_s[tmp$cluster==k,]),pch=20,
                             ylim=c(0,1000000))}
  else{plot(x_s[tmp$cluster==k,],pch=20,ylim=c(0,1000000))}
}
```



Kmeans with Euclidean distance

Representation of the cluster means

```
par(mfrow=c(2,2))  
for (k in 1:4){  
  plot(tmp$centers[k,],type='l',ylim=c(0,1000000))  
}
```



Dynamic Time Warping

The Euclidean distance is influenced by non-alignment of time series:

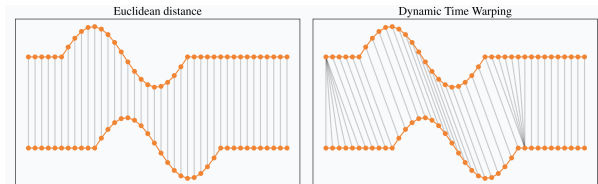


Figure from <https://rtavenar.github.io/blog/dtw.html>

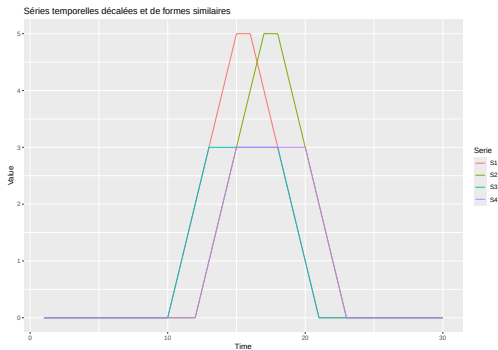
Dynamic Time Warping look for the best alignment of the 2 time series.

A distance can be build by measuring the distance between the best time series alignment.

```
library(dtw)
distMatrix <- dist(x_s, method="DTW")
```

Dynamic Time Warping

Comparison of DTW and Euclidean distances on a simulated example:



Dynamic Time Warping

Comparison of DTW and Euclidean distances on a simulated example:

```
## euclidean distance:
```

```
##           1           2           3
```

```
## 2 6.000000
```

```
## 3 3.162278 5.656854
```

```
## 4 5.656854 3.162278 4.472136
```

```
## DTW distance:
```

```
##      [,1] [,2] [,3] [,4]
```

```
## [1,]    0    0    6    6
```

```
## [2,]    0    0    6    6
```

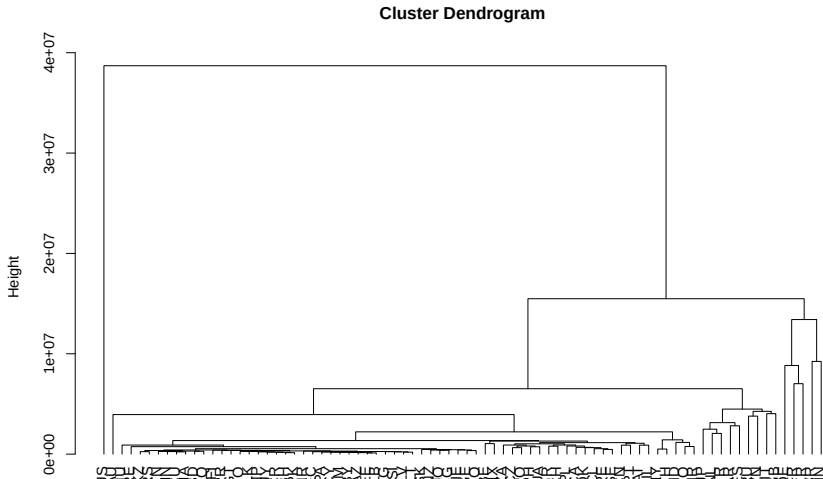
```
## [3,]    6    6    0    0
```

```
## [4,]    6    6    0    0
```


Hierarchical clustering

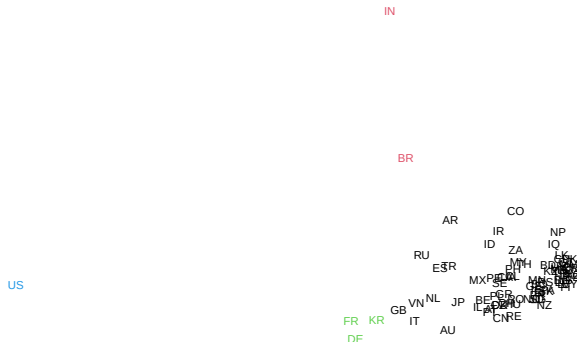
Lets do back to Covid time series. We can then apply any clustering algorithm using this DTW distance

```
hc <- hclust(distMatrix, method="average")  
plot(hc, hang = -1)
```



Clustering representation

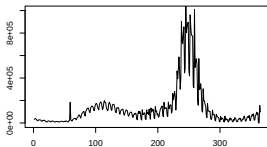
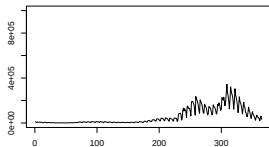
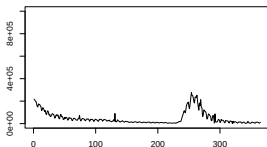
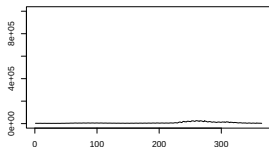
```
cluster=cutree(hc,4)
mds2 <- cmdscale(distMatrix)
plot(mds2, type="n", axes=FALSE, ann=FALSE)
text(mds2, labels=rownames(x_s), xpd = NA,col = cluster)
```



Clustering representation

Representation of the cluster means

```
par(mfrow=c(2,2))
for (k in 1:4){
  if (sum(cluster==k)>1){
    plot(colMeans(x_s[cluster==k,]),type='l',xlab='',ylab='',ylim=c(0,1))
  } else{plot(x_s[cluster==k,],type='l',xlab='',ylab='',ylim=c(0,1))
}
```

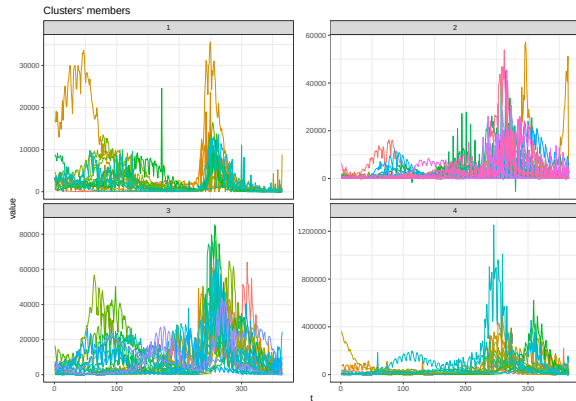


To go further

The dtwclust package

The following package allows different type of clustering, based on DTW distances:

```
library(dtwclust)
tmp=tsclust(x_s,k=4)
plot(tmp)
```

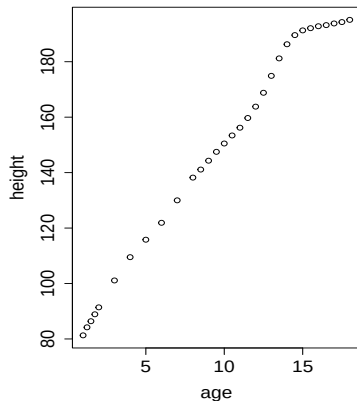


Functional data approach

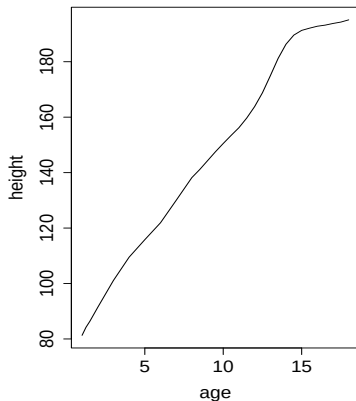
An alternative way to work with time series is to assume that $x_i(t_1), \dots, x_i(t_m)$ are **discrete observations of a function**:

$$x_i(t) \quad \text{with } t \in [0, T]$$

discrete observations



latent function



This is the **functional data approach**

Functional data approach

The advantages of the functional data approach vs the usual multidimensional approach:

- ▶ parsimonious modelling of the curves
- ▶ allows to deal with irregularly sampled time series

The funHDDC package

The funHDDC package provides clustering algorithm for functional data

```
library(funHDDC)
basis<- create.bspline.basis(c(1,365), nbasis=25)
var<-smooth.basis(argvals=seq(1,365,length.out =365),
                  y=t(x_s),fdParobj=basis)$fd
res<-funHDDC(var,K=1:4,init='random')
```

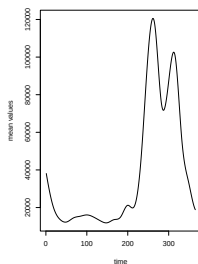
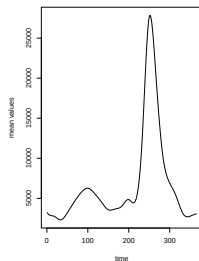
```
## funHDDC:
```

```
##          model K threshold complexity          BIC          co
## 1 AKJBKQKDK 2          0.2          103 -62,356.33
## 2 AKJBKQKDK 1          0.2           75 -236,542.16
## 3 AKJBKQKDK 3          0.2          <NA>      -Inf pop<min.indivi
## 4 AKJBKQKDK 4          0.2          <NA>      -Inf pop<min.indivi
##
## SELECTED: model AKJBKQKDK with 2 clusters.
## Selection Criterion: BIC.
```


The funHDDC package

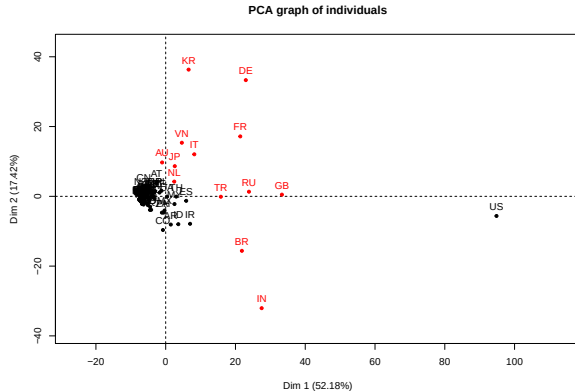
Representation of the cluster means

```
par(mfrow=c(1,res$K))  
for (k in 1:res$K){  
  plot(mean.fd(fd(var$coefs[,which(res$class==k)],  
                  var$basis)))  
}
```



Clustering representation

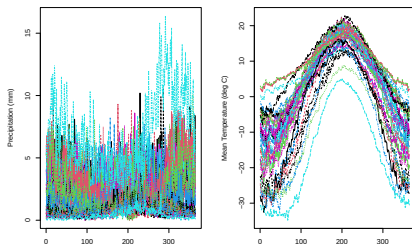
```
plot(pca,choix = "ind",col.ind = res$class,  
     graph.type = "classic")
```



Exercise

Carry out a clustering of Canadian weather stations based on precipitation, then temperatures.

```
library(fda)
data("CanadianWeather")
par(mfrow=c(1,2))
matplot(day.5, CanadianWeather$dailyAv[, , "Precipitation.mm"],
        type="l", xlab="", ylab="Precipitation (mm)")
matplot(day.5, CanadianWeather$dailyAv[, , "Temperature.C"],
        type="l", xlab="", ylab="Mean Temperature (deg C)")
```



Is there a link with the geographical location of the cities (Atlantic, Pacific, Continental, Arctic), available in the variable *region*?

Features extraction from time series

Features extraction

Another ways to work with time series is to extract features from them:

```
library(tsfeatures)
x=CanadianWeather$dailyAv[, , "Temperature.C"]
xf=tsfeatures(x)
print(dim(xf))
```

```
## [1] 35 16
```

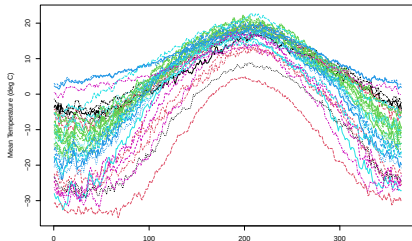
```
summary(xf)
```

```
##      frequency      nperiods seasonal_period      trend
## Min.      :1    Min.      :0    Min.      :1      Min.      :0.9907    Min
## 1st Qu.:1    1st Qu.:0    1st Qu.:1      1st Qu.:0.9947    1st
## Median :1    Median :0    Median :1      Median :0.9959    Med
## Mean    :1    Mean    :0    Mean    :1      Mean    :0.9954    Mea
## 3rd Qu.:1    3rd Qu.:0    3rd Qu.:1      3rd Qu.:0.9964    3rd
## Max.     :1    Max.     :0    Max.     :1      Max.     :0.9984    Max
##      linearity      curvature      e_acf1      e_acf10
## Min.      :2.736    Min.      :-18.22    Min.      :0.4588    Min.      :0.
## 1st Qu.:4.048    1st Qu.: -17.76    1st Qu.:0.5071    1st Qu.:0.
## Median :5.307    Median : -17.27    Median :0.5684    Median :0.
```

Features extraction

and then we can for instance do a clustering on the basis of these features :

```
library(mclust)
xclus=Mclust(xf,G=1:8)
matplot(day.5, CanadianWeather$dailyAv[, , "Temperature.C"],
        type="l", xlab="", ylab="Mean Temperature (deg C)",
        col=xclus$classification)
```



Clustering result for Canadian Weather

```
library(ggplot2)
library(maps)
data(CanadianWeather)
cities <- data.frame(
  city = CanadianWeather$place,
  lon = -CanadianWeather$coordinates[,2],
  lat = CanadianWeather$coordinates[,1]
)
map_canada <- map_data("world", region = "Canada")
ggplot() +
  geom_polygon(data = map_canada, aes(x=long,y=lat,group=group),
    fill = "lightgray", color = "black") +
  geom_text(data = cities, aes(x = lon, y = lat, label = city),
    color = xclus$classification, size = 3) +
  coord_fixed(1.3) +
  labs(title = "Clustering des villes en fonction de leur
    courbe de température") +
  theme(plot.title = element_text(hjust = 0.5))
```

Clustering result for Canadian Weather

Carte du Canada avec les noms des villes du dataset CanadianWeather

